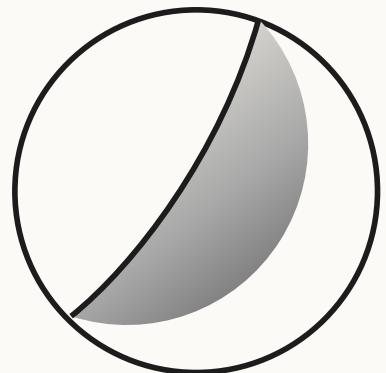


PORTFOLIO

Pim

I build software that takes work off your hands.



More work and contact: github.com/pimdaanbram-prog

About me

I'm Pim. I build software that takes work off your hands: web apps, desktop software and tools for tasks that come back every day.

I always start from a real problem, often my own. A training app I still use every week, a desktop app that lets a team share files without losing work, a tool that lets a contractor write a quote in a few minutes.

BUILDS

Web apps, desktop software and automations

FOR

Mac, Windows, phone and browser

What I can build for you

Web apps people actually use

An app that works on phone and laptop, even without internet, and installs like a regular app. Fast, clear and in your language.

See: Strength Tracker

Software for teams

Desktop software for Mac and Windows that makes working together easier, with secure cloud storage and without anyone losing work.

See: TeamSync

Tools that take over boring work

Does something come back every day, like writing a quote or filling in a form? Then I build a tool that does it in a fraction of the time.

See: OfferteVlot, Belhulp

AI where it pays off

Not a chatbot for the sake of it, but AI where it saves time: transcribing calls, filling in data, turning images into print. With checks, so it is right.

See: Belhulp, Kleurboek

Contents

<i>I</i>	<i>TeamSync</i>	Desktop app for Mac and Windows
<i>II</i>	<i>Strength Tracker</i>	Installable web app
<i>III</i>	<i>OfferteVlot</i>	Web app for contractors
<i>IV</i>	<i>Belhulp</i>	Call assistant for the Mac
<i>V</i>	<i>Kleurboek</i>	Print: a coloring book for Amazon
<i>VI</i>	<i>Solana Forensics</i>	Research tool for blockchain data

TeamSync

DESKTOP APP FOR MAC AND WINDOWS · DELIVERED AND IN USE

One shared folder for a whole team, without anyone ever overwriting someone else's work.

Who it is for

A small team on both Macs and Windows laptops that works on the same files.

The problem

Sending files back and forth always goes wrong at some point: someone works in an old version and someone else loses their work. Real co-editing through Microsoft 365 needs a licence per person and an administrator who signs off, which a small team usually does not have.

What I built

I built a desktop app where the cloud is the source of truth and every laptop keeps its own copy. A custom sync engine compares, per file, the local version, the last shared version and the cloud version, and picks exactly one action. When two changes collide, both are kept. The team sees who has a document open and can also write in the same document at the same time.

What it delivers

1. No more overwritten work: on a collision both versions are kept.
2. Version history per file, an older version can always be restored.
3. You see who has a Word document open before you start on it.
4. Write together at the same time and save the result as a Word file again.
5. Runs on Mac and Windows and updates itself automatically.

Under the hood

- a. A Rust sync engine whose decision rules can be tested on their own: every situation has its own test.
- b. Access rules at database level for every table and for storage, with tests that check them.
- c. Integration tests that simulate two laptops, including offline work, renames and conflicts.

The hardest part

Edits in Word were sometimes silently lost: while Word had a document open, the file looked unchanged and was never uploaded. I taught the app to treat open documents differently and wrote a test that replays opening, editing, saving and closing exactly, so it cannot come back.

Built with

Tauri 2 · Rust · React · TypeScript · Supabase · Yjs

Strength Tracker

INSTALLABLE WEB APP · LIVE AND USED EVERY WEEK

My own training app: log workouts on phone and laptop, even without signal, with concrete advice every week.

Who it is for

My training partner and me. I still use it every week.

The problem

Training apps are often English-only, work poorly without internet, or do not let two people log at the same time. I wanted one app that shows the same on phone and laptop, simply works in a gym without signal, and tells me after each week which weight to use next time.

What I built

Strength Tracker is a web app you install like a regular app. You log sets, weights and reps, alone or together with your training partner. Everything is stored on your own device first and then syncs to your other devices by itself. It includes a library of over a hundred exercises, a 3D figure that shows which muscles you trained this week, and weekly advice for your next weights.

What it delivers

1. Works offline in the gym and syncs as soon as there is signal again.
2. Train together: several profiles in one session.
3. Concrete weight advice every week, based on the week before.
4. Installable on phone and laptop, straight from the browser.

Under the hood

- a. Data stored locally first, then synced in real time between devices.
- b. A custom service worker, so the app also opens without internet.
- c. Rebuilt in ten steps into a fast app that only loads what you need, with 110 automated tests.

The hardest part

On a new device the app stayed empty while the data did arrive: the active profile pointed to a profile that only existed locally. Now the app picks a valid profile automatically after merging, and shows a clear message when syncing gets stuck somewhere.

Built with

React · TypeScript · Vite · Tailwind CSS · Supabase · Three.js

Links

Open the app: strengttracker.netlify.app

Source on GitHub: github.com/pimdaanbram-prog/strength-tracker

III *OfferteVlot*

WEB APP FOR CONTRACTORS · READY TO USE, NO SERVER NEEDED

A clean quote as a PDF in a few minutes, with your own logo and the VAT calculated correctly.

Who it is for

Contractors and roofers who want to send a quote quickly after a visit or a phone call.

The problem

After an inspection or a phone call you want to send a clean quote straight away, with your own logo and a correct VAT calculation. Without creating an account or installing software first.

What I built

OfferteVlot runs entirely in the browser. You enter your company details, the customer and the line items, and meanwhile you see exactly what the PDF will look like. No account and no server: it is a single file you simply open, and your data stays on your own device.

What it delivers

1. A quote ready in a few minutes.
2. Your own logo and details on a sharp A4 PDF.
3. VAT and discounts calculated correctly, automatically.
4. Send it straight away via WhatsApp with a neat cover message.
5. Sequential quote numbers and a list of earlier quotes.

Under the hood

- a. Custom PDF layout with pagination and a table header that repeats on every page.
- b. The live preview and the PDF use the same calculation rules, so they always match.
- c. Every logo, SVG included, is placed sharply in the PDF.

The hardest part

The total page count is only known once everything is drawn. The first version therefore left a gap in the footer. Now a final pass stamps 'Page X of Y' on every page, and a table that runs across two pages gets its header back automatically.

Built with

React · TypeScript · Vite · Tailwind CSS · jsPDF

Links

Source on GitHub: github.com/pimdaanbram-prog/Whatsapp-analyser

IV Belhulp

CALL ASSISTANT FOR THE MAC · IN USE DURING CALLS

Listens in on sales calls, transcribes live and fills in the call form by itself.

Who it is for

Myself, for the sales calls I make for school.

The problem

During a sales call you have to listen, ask follow-up questions and keep a form up to date, all at once. Typing it up afterwards takes time, and names, phone numbers and agreements get lost or end up wrong.

What I built

Belhulp runs on my Mac. My voice and the customer's are recorded separately and transcribed straight away. From the conversation it fills in the questions of the call form, keeps every call in an overview, and uses a browser extension to put the answers into the system I work in.

What it delivers

1. No more typing up after a call.
2. Names, numbers and agreements end up correct.
3. Every call is neatly kept in one overview.
4. Without the customer's consent nothing is recorded or stored.

Under the hood

- a. Fast speech recognition on the Mac itself for the live view, and a more accurate model that corrects every part of the call afterwards.
- b. Fields are filled through a fixed schema with an 'unknown' option, so nothing is guessed.
- c. Every call goes to a queue first, so none is ever lost.

The hardest part

Bits of audio dropped out whenever speech recognition claimed the processor, which mangled names. I separated recording from processing, so the recording never has to wait. After that no audio was lost.

Built with

Python · faster-whisper · Gemini · Flask · Swift · Chrome-extensie

v *Kleurboek*

PRINT: A COLORING BOOK FOR AMAZON · READY FOR PRINT

A print-ready coloring book of fifty plates, with an own workflow that turns AI drawings into clean lines and inspects every plate.

Who it is for

Readers who like calm, cozy coloring pages, via Amazon and a Dutch edition.

The problem

Selling a coloring book takes more than nice pictures. AI coloring books are rightly criticised for grey pixels, solid black areas, tiny details and drawings that run into the page edge, and Amazon has strict rules for the interior and the cover.

What I built

I built a workflow that creates the drawings, cleans them up and turns them into vector lines at real print size. Every plate is inspected automatically, and then the interior, the cover and the sales images are built, each with its own check before it goes to the printer.

What it delivers

1. Fifty plates with the same five characters, recognisable every time.
2. No grey pixels and no solid areas: everything prints as clean lines.
3. A Dutch edition from the same plates, without drawing anything again.

Under the hood

- a. Automatic inspection per plate: line weight, regions that are too small and shapes that float free.
- b. A cleanup step from raw image to vector lines at print size.
- c. Own checks for interior and cover, tested with deliberately broken files.

The hardest part

The characters had to look the same on all fifty plates. Every request therefore came with a fixed reference sheet, and I adjusted the requests based on test failures, such as a hedgehog without a snout or a cat without whiskers.

Built with

Python · OpenCV · scikit-image · potrace · PyMuPDF · OpenAI Image API

VI *Solana Forensics*

RESEARCH TOOL FOR BLOCKCHAIN DATA · WORKING RESEARCH TOOL

Reconstructs what really happened in a meme-coin launch, read-only and with an evidence grade per conclusion.

Who it is for

Research into scams around new tokens.

The problem

After a scam with a new token you want to know exactly what happened: who created it, who sold and where the money went next. Ordinary overviews get confused by detours through other tokens and by fake addresses, and produce conclusions that look convincing but are wrong.

What I built

I built a command-line tool that analyses transactions, wallets and complete launches and shows the results on a local dashboard. Groups of wallets that work together get an evidence grade that is fixed in code. The tool holds no key, signs nothing and never trades.

What it delivers

1. A timeline from creation to sale, with the evidence at every step.
2. Groups of cooperating wallets, each with an honest evidence grade.
3. Read-only: no keys, no signatures, no trading.

Under the hood

- a. Trades are derived directly from the trading pool itself, so detours cannot distort the price.
- b. Recognises pools by the program that manages them, for several trading platforms at once.
- c. Not a single external dependency: only what Node.js ships with.

The hardest part

Fake addresses that look almost identical to real ones (address poisoning) inflated groups of wallets artificially. Those relationships now get the lowest evidence grade or are excluded entirely, so a conclusion never rests on such a trick.

Built with

Node.js · Solana RPC · WebSocket · SVG